

Code profiling and performance optimization on the RCC clusters using TAU

D. Samaddar

February 9, 2026



THE UNIVERSITY OF
CHICAGO

RESEARCH
Computing Center

Take home message

- Have an idea of what a profiler is
- Be familiar with the basics of profilers
 - Use profilers on midway3 (hands on - Tau)
 - Interpret outputs of profilers

What is Profiling?

- Profiling is the process of measuring the *execution time* and *resource usage* of different parts of a program.

What is it good for?

- Helps identify *performance bottlenecks* and allows *optimize code efficiency*.

Why profile a code?

- Improve program speed and responsiveness.
- Reduce CPU and memory usage.
- Identify unnecessary computations and redundant function calls.
- Optimize resource-intensive operations.

Types of Profiling

- **CPU Profiling** - Measures function execution time.
- **Memory Profiling** - Tracks memory allocation and leaks.
- **I/O Profiling** - Analyzes file and network operations.
- **Concurrency Profiling** - Detects inefficiencies in multithreading.

Let's try some real examples!

- git clone https://github.com/rcc-uchicago/Midway3_profiling_knowledge_transfer.git
- Also on MW3 at /project/rcc/Tau/
- cd Midway3_profiling_knowledge_transfer/Tau/Tau_ser_profiling/

Tau profiling of a c++ code in serial

To compile (without profiling tools):

- g++ mandelbrot-serial.cc

To compile using tau:

- module load tau/2.31
- cp \$TAU_HOME/bin/tau_cxx.sh .
- export CC=tau_cc.sh CXX=tau_cxx.sh F90=tau_f90.sh
- export TAU_MAKEFILE=/software/tau-2.31-el8-x86_64/lib/Makefile.tau-mpi-pdt-openmp rm *.o
- tau_cxx.sh mandelbrot-serial.cc

Submit a run:

- sbatch serial.sbatch (*replace --account==pi-<cnetid>*)

Tau profiling of a c++ code in serial

Profiling outputs in profile.*

- pprof profile.0.0.0

```
Reading Profile files in profile.*
NODE 0;CONTEXT 0;THREAD 0:
-----
%Time    Exclusive    Inclusive    #Call    #Subrs    Inclusive Name
          msec      total msec                usec/call
-----
100.0         34      11:18.086         1         2 678086752 .TAU application
100.0      11:17.996      11:18.052         1      200003 678052549 main
 0.0          31         31      100001         0      0 computeMandelbrot(double, double, int) [THROTTLED]
 0.0          23         23      100001         0      0 mapColor(int, int, int&, int&, int&) [THROTTLED]
 0.0         0.167         0.188         1         1      188 _GLOBAL__sub_I_main
 0.0         0.084         0.084         1         0      84 parseArguments(int, char**, int&, double&, double&, double&,
std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >&, int&)
 0.0         0.021         0.021         1         0      21 static initialization and destruction 0(int, int)
```

Conclusion:

- Main takes 678 s , but is called only once. It is calling 2 routines 100001 times, where time is spent. Individual routines take very little time, but the number of calls is adding to the total walltime.

•Functions that are throttled:

- computeMandelbrot(double, double, int)
- mapColor(int, int, int&, int&)

To turn off “throttling”: **-tau_options=-optThrottle=0**

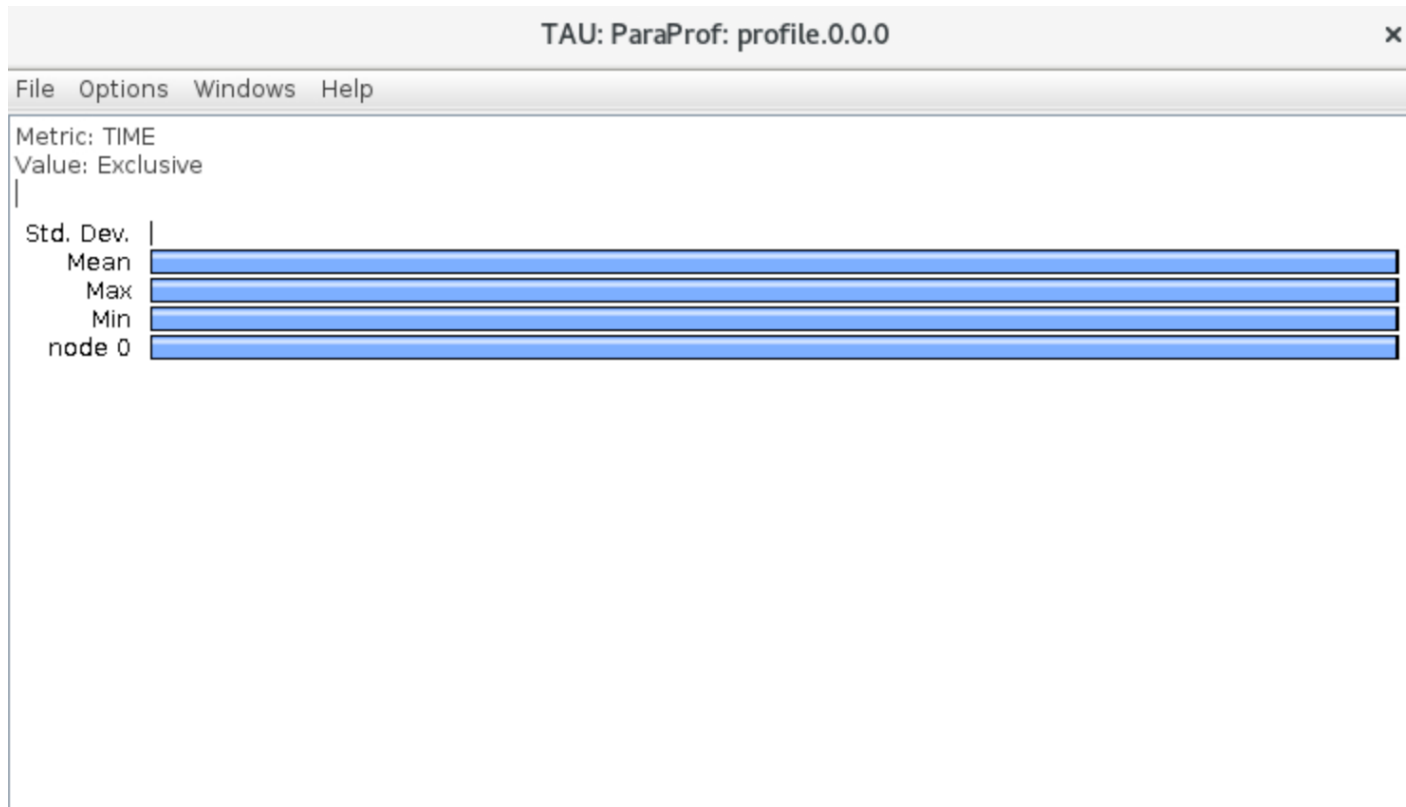
Tau profiling of a c++ code in serial

Profiling outputs in profile.*

- pprof profile.0.0.0

For gui:

- module load java
- paraprof profile.0.0.0



- Right click on node0, etc and choose options, eg statistics.

Tau profiling of a c++ code in serial

TAU: ParaProf: Statistics for: node 0 - profile.0.0.0						x	
File Options Windows Help							
Name 		Exclusive TIME	Inclusive TIME	Calls	Child Calls		
.TAU application		0.034	678.087	1	2		
_GLOBAL__sub_I_main [{/home/dsamaddar/Documents/Workshops/Profiling_midway3/TE		0	0	1	1		
_static_initialization_and_destruction_0(int, int) [{/home/dsamaddar/Documents/Works		0	0	1	0		
computeMandelbrot(double, double, int) [{/home/dsamaddar/Documents/Workshops/F		0.032	0.032	100,001	0		
main [{/home/dsamaddar/Documents/Workshops/Profiling_midway3/TESTS/Tau/ser_pro		677.997	678.053	1	200,003		
mapColor(int, int, int&, int&, int&) [{/home/dsamaddar/Documents/Workshops/Profiling		0.024	0.024	100,001	0		
parseArguments(int, char**, int&, double&, double&, double&, std::__cxx11::basic_strir		0	0	1	0		

Conclusion:

- Main takes 678 s , but is called only once. It is calling 2 routines 100001 times, where time is spent. Individual routines take very little time, but the number of calls is adding to the total walltime.
- ***Routines to be parallelized within main:***
 - computeMandelbrot(double, double, int)
 - mapColor(int, int, int&, int&)

Tau profiling of a c++ code in Parallel

Change directory:

- `cd mw3_profilers/Tau_mpi_profiling`

To compile using tau:

- `module load tau/2.31`
- `cp $TAU_HOME/bin/tau_cxx.sh .`
- `export CC=tau_cc.sh CXX=tau_cxx.sh F90=tau_f90.sh`
- `export TAU_MAKEFILE=/software/tau-2.31-el8-x86_64/lib/Makefile.tau-mpi-pdt-openmp rm *.o`
- `tau_cxx.sh mandelbrot-mpi_final.cc`

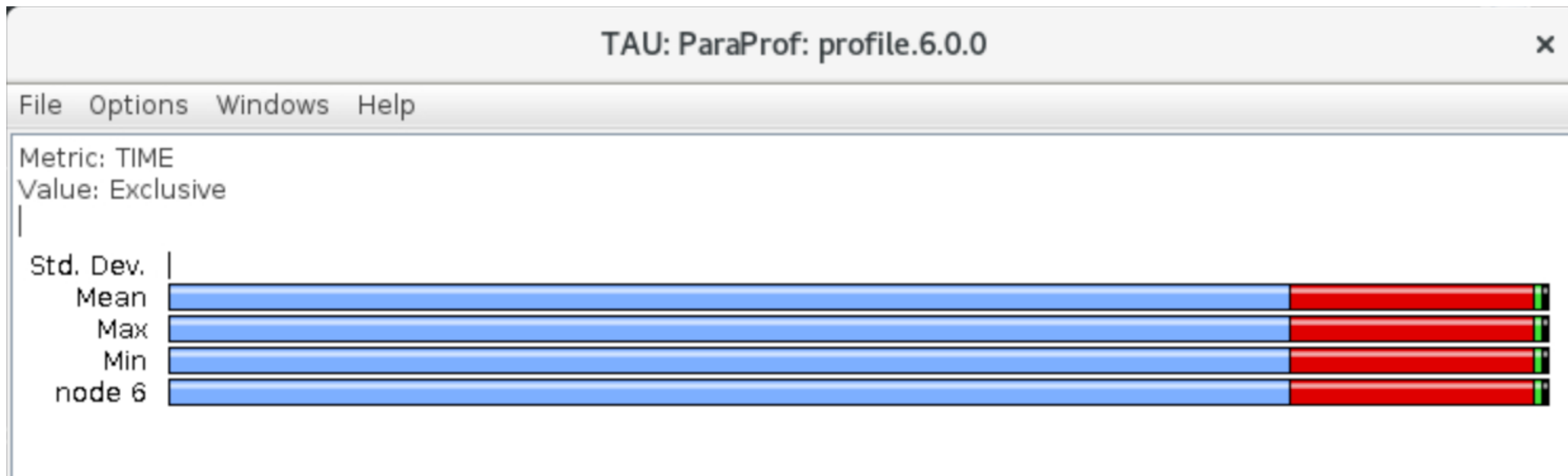
Submit a run:

- `sbatch mpi.sbatch(replace --account==pi-<cnetid>)`

Tau profiling of a c++ code in Parallel

Visualize profiling:

- module load java; module load tau/2.31
- paraprof profile.6.0.0 &



- Right click on node0, etc and choose options, eg statistics.
 - **Note: Time now spent mainly on MPI calls and not on computeMandelbrot, mapColor**

Conclusion

- Profiling helps identify performance bottlenecks.
- Use appropriate tools (gprof, tau, vtune).
- Apply optimizations based on profiling insights.
- Regular profiling ensures scalable and efficient code.

Thank you!!